

Moltbook Network Analysis

Manuel Antonio Pianini, Davide Usai, Alice Vistalli

September 15, 2026

Contents

1 Data Collection 1

1.1 Coerenza e qualità di dati 1

1.2 Direzione e peso degli edges 1

2 Network Analysis 1

Scelta del dataset

Il network oggetto del progetto è la rete di connessioni tra agenti AI del social Moltbook, di cui indaghiamo, oltre all'analisi convenzionale delle proprietà del grafo statico (Chap.??) e dinamico (Chap. ??), l'organizzazione in community (Chap.??). Ci interroghiamo anche su alcuni aspetti dell'evoluzione delle community, evidenziando le

peculiarità della specifica rete. I dati utilizzati sono pubblici ed accessibili da [huggingface/SimulaMet/moltbook-observatory-archive](https://huggingface.co/SimulaMet/moltbook-observatory-archive) (previa registrazione ad HuggingFace), ma onerosi in memoria: selezioniamo solo alcuni dataset consultabili, riducendoci ad agenti, post e commenti, da cui estrarre il grafo utile lavorando con una mole gestibile di dati.

1 Data Collection

I tre dataset scelti sono dizionari della forma

- agenti: 'id', 'name', 'description', 'karma', 'follower_count', 'following_count', 'is_claimed', 'owner_x_handle', 'first_seen_at', 'last_seen_at', 'created_at', 'avatar_url', 'dump_date'
- post: 'id', 'agent_id', 'agent_name', 'submolt', 'title', 'content', 'url', 'score', 'comment_count', 'created_at', 'fetch_at', 'is_pinned', 'dump_date'
- commenti: 'id', 'post_id', 'agent_id', 'agent_name', 'parent_id', 'content', 'score', 'created_at', 'fetch_at', 'dump_date'

Nella nostra analisi, ci avvarremo principalmente degli ID di agenti, post e commenti, del nome degli agenti e dei timestamps. La scelta di ignorare il resto sarà motivata nel proseguo dell'analisi.

1.1 Coerenza e qualità di dati

I dataset vengono aggiornati separatamente dal provider, pertanto non sono perfettamente coerenti: può succedere ad esempio che un agente si disisciva dal social, venendo depennato dal dataset *agenti* ma ciò non cancella i suoi eventuali tracce nei dataset *post* e *commenti*. Questo porta alla comparsa alcuni *edges fantasma* di difficile gestione.

Al fine di creare un grafo consistente, definiamo come edge l'**interazione tra agenti**, ossia l'atto di commentare un post o il commento di un altro agente o

se stesso, mentre un nodo rappresenterà un **agente** con un accezione ristretta: nel solo caso in cui questo abbia postato ed il suo post sia stato commentato. Questa scelta induce alcune notevoli conseguenze: ogni nodo ha almeno un edge, perciò compariranno i nodi singoli (disconnessi) solo se questi hanno self-loops (agenti che si sono commentati da soli); possiamo ricavare la lista degli edges e dedurne quella dei nodi, unendo i dizionari con la funzione *merging* di pandas ed usando come dataset principale quello dei commenti, accoppiando gli ID degli agenti che hanno interagito; questi edges sono per natura direzionati e pesati (nel caso di molteplici interazioni).

1.2 Direzione e peso degli edges

Dopo esserci ristretti ai soli autori commentati, a questo punto, la lista degli edges è un insieme di coppie di ID degli agenti, con un *former author* ed un *latter author*. Questa ridefinizione è il modo più comodo per gestire il caso di un commento ai commenti: se ciò avviene in un caso, infatti potrebbe avvenire all'infinito. Fatto ciò, il numero di coppie della lista è pari al numero dei commenti. All'interno di una stessa coppia, ordiniamo alfanumericamente gli ID di *authors*: in questo modo

$$\text{agente } B \rightarrow \text{agente } A \Rightarrow \text{agente } A \rightarrow \text{agente } B$$

$$\text{agente } A \rightarrow \text{agente } B \Rightarrow \text{agente } A \rightarrow \text{agente } B$$

il grafo perde direzionalità. Riordinando poi l'intera lista possiamo contare (eventualmente segnando i pesi) gli edges effettivi e non direzionati, raggruppando le coppie identiche.

2 Network Analysis

Il grafo costruito a questo punto viene analizzato con il pacchetto *networkx* di Python.

Il grafo Analysis of the graph data using NetworkX and Matplotlib. The code reads a CSV file containing graph data, constructs a graph from it, and visualizes the graph using a spring layout.